SQL Lab 1 – The Basics (requires access to the samples.high_peaks table)

In order to complete this lab, you will need to connect to the class database (INFO4555) via the SQL Developer application found on your AppStream. Details on how to connect to the class database may be found in document: *"Connecting to class database via SQL Developer",* in the Project Files folder of the class Canvas site.

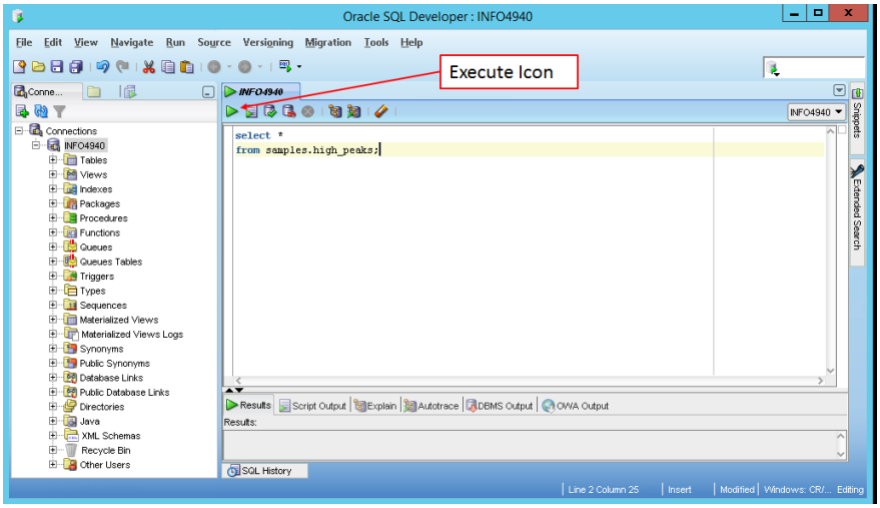
For this lab we will use a single table named "high_peaks", which is located in the "samples" schema of the INFO4555 database. The HIGH_PEAKS table contains a record for each of the 46 High Peaks of the Adirondack mountains of New York state.

HIGH_PEAKS		
Column Name	**Data Type**	**Description**
Peak_Rank	Number	Rank (1-46) of the peak by elevation
Peak_Name (PK)	Character	Name of Peak as designated by the 46ers club. This is also a unique identifier for this table.
Elevation	Number	Elevation of Peak measured in feet.
Ascent_of_climb	Number	The Total vertical ascent measured in feet.
Difficulty	Number	Difficulty rating (1-7) 1 being easiest, according to Adirondack.net
Round_Trip_mileage	Number	Round trip mileage of hike, according to Adirondack.net
Typical_Hike_Time_Hrs	Number	Typical hike time (round trip) in hours, according to Adirondack.net

A query must have both a SELECT and FROM clause. We'll start with a simple query:

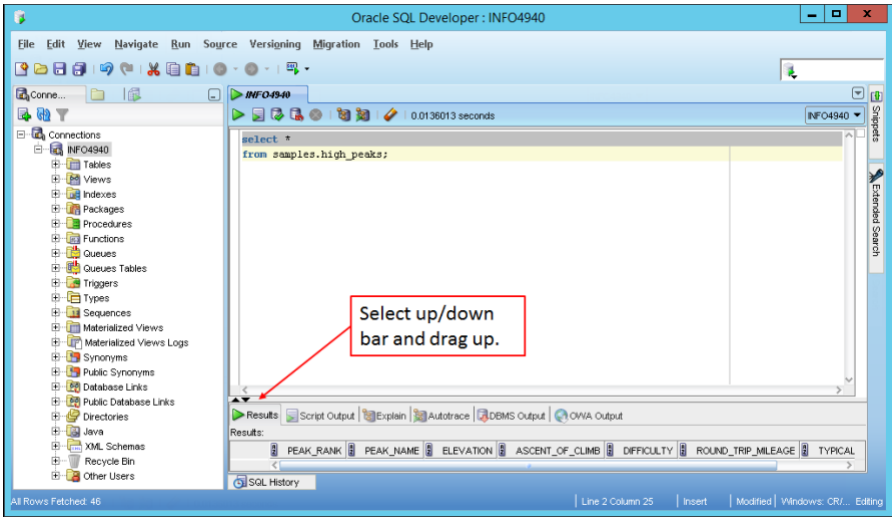
After connecting to the INFO4555 database with SQL Developer, type the following into the SQL edit window and click the execute icon (Green arrow).

```
SELECT *
FROM samples.high_peaks;
```

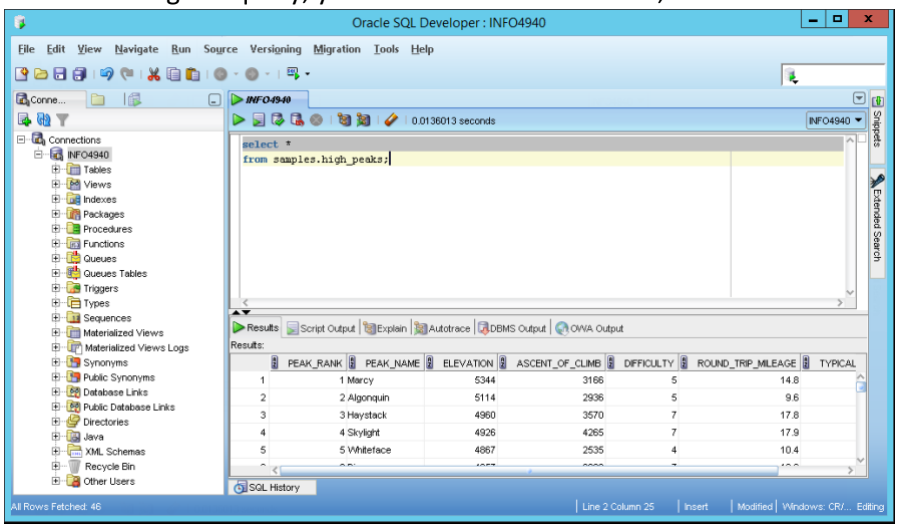


SQL Lab 1 – The Basics (requires access to the samples.high_peaks table)

**** Note. If you don't see the query result set, after clicking the query execute icon, you may need to expand the result set window. To expand the result set window, click on the up/down bar and drag upward. You will see the result set below.*



After executing the query, you should see the result set, similar to below:



Notice the scrollbar to the right of the result set. You may scroll through the 46 records.

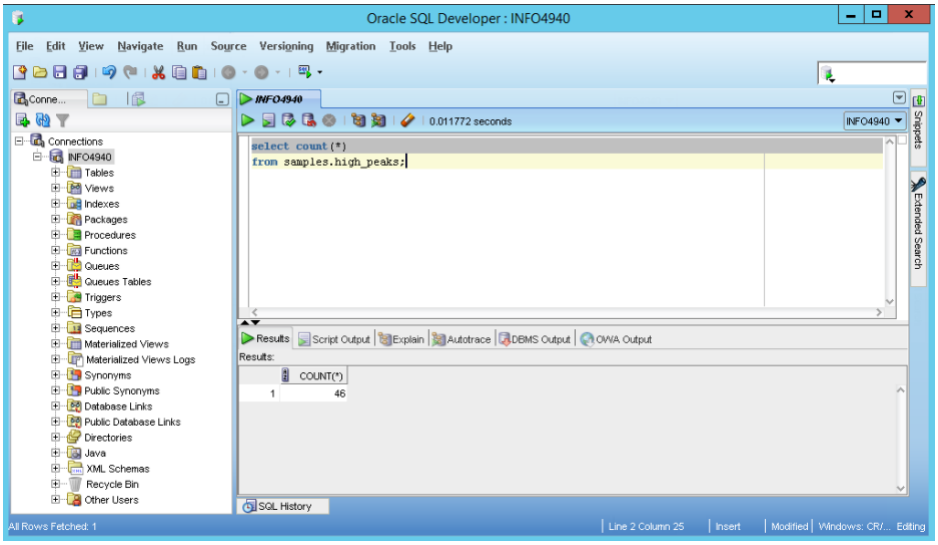
In the first query we specified a wildcard '*' for our SELECT clause, resulting in all columns being returned. For the FROM clause we specified the table being queried high_peaks and qualified the table name with the schema name it is located in, **samples.**high_peaks.

Next we will try some basic database functions in the SELECT clause.
Edit your original query by adding a "count()" function to the SELECT clause.

```
SELECT count(*)
FROM samples.high_peaks;
```

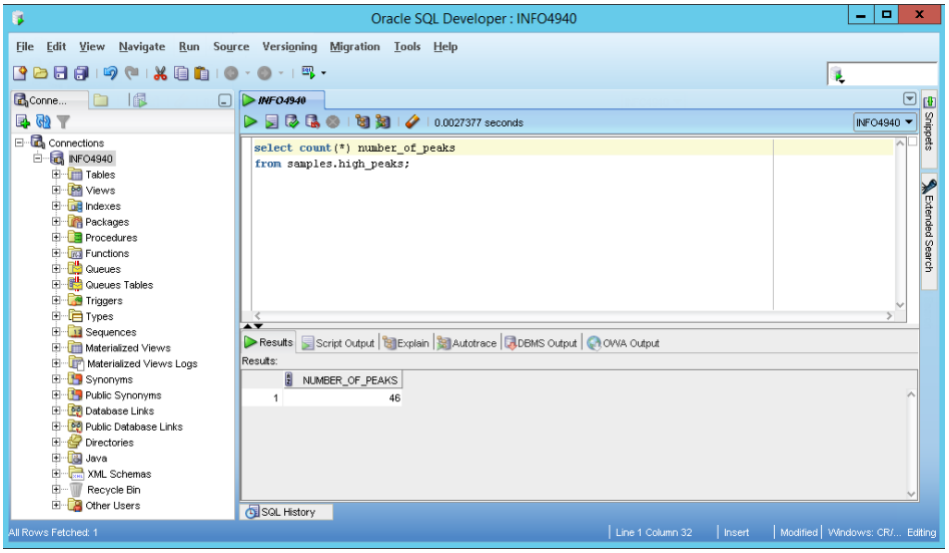
Execute your new query.

SQL Lab 1 – The Basics (requires access to the samples.high_peaks table)



Notice the result set is now a single line: the number of records found in the table, 46. Also, notice the label for this column, it is simply the function, "COUNT(*)".

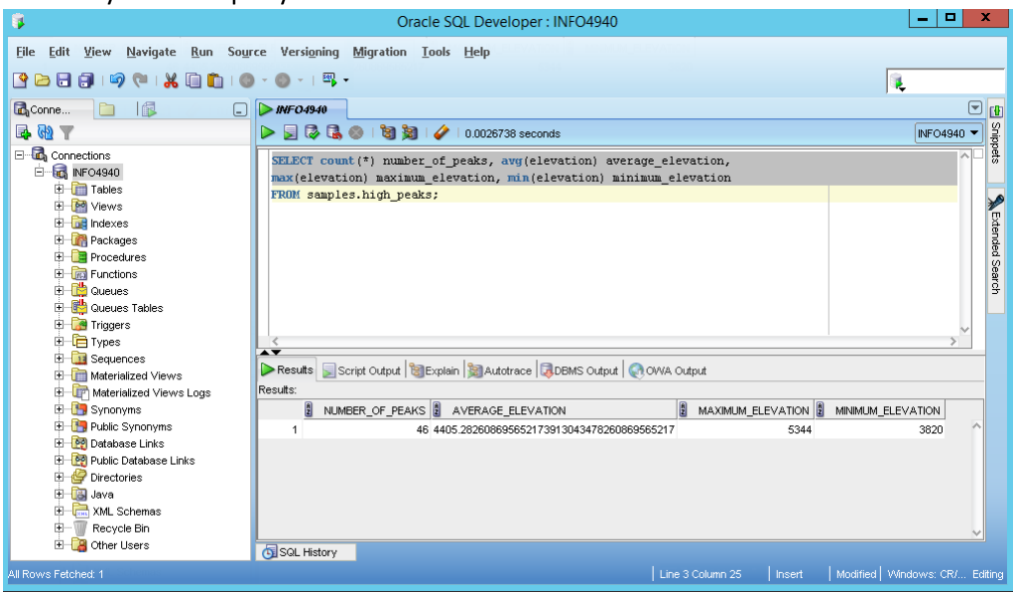
This time we will assign an alias for this derived column, something with more meaning, such as "number_of_peaks".
Edit your SQL, by adding the alias, and re-execute your query.
SELECT count(*) **number_of_peaks**
FROM samples.high_peaks;



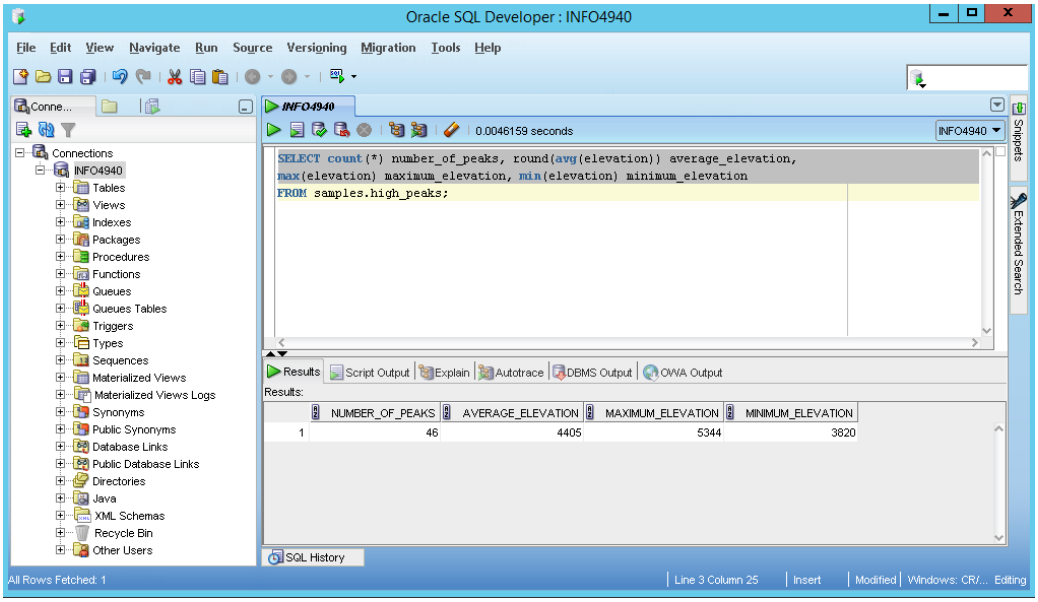
This time we will try a few more common SQL functions, and give the columns nice aliases.
Edit your SQL statement again, by adding a few more functions to the SELECT clause:
SELECT count(*) number_of_peaks, avg(elevation) average_elevation,
max(elevation) maximum_elevation, min(elevation) minimum_elevation
FROM samples.high_peaks;

SQL Lab 1 – The Basics (requires access to the samples.high_peaks table)

Execute your new query.



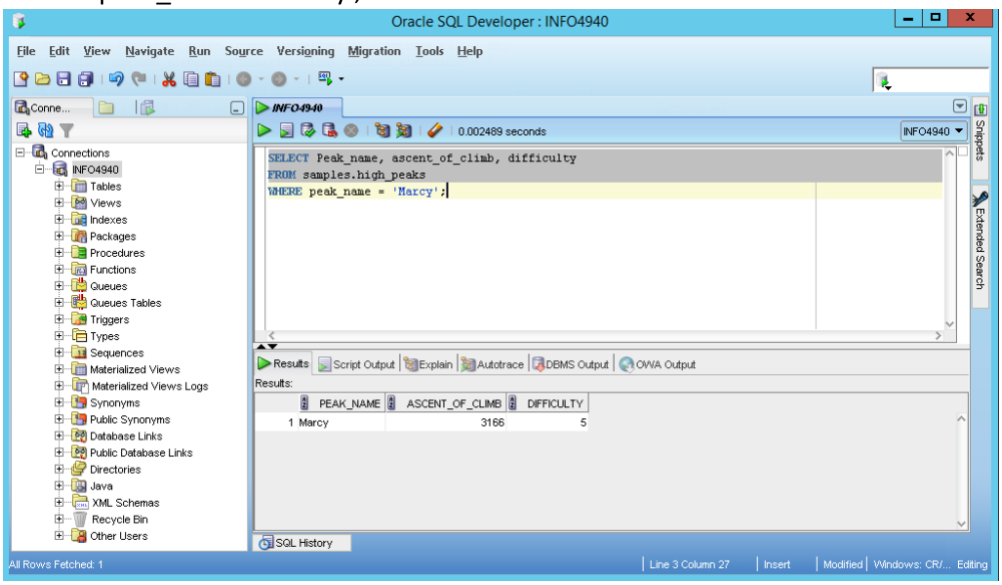
You have now produced a nice summary of the dataset. Number_of_peaks, average_elevation, maximum_elevation, and minimum_elevation. You will notice the average elevation goes to a precision of 36 decimal places. Yikes! This looks messy, and we really only need the average to the nearest whole number. So, let's round the result to the nearest whole number. We can call functions from functions. Edit your query and wrap the avg() function with the round() function.



Now we will introduce the WHERE clause. The WHERE clause is used for filtering the result set and joining tables. Since we are only using a single table for this lab, we will focus on filters.
First we will change our SELECT clause to only three attributes from the table: peak_name, ascent_of_climb, and difficulty. Then add a WHERE clause explicitly identifying Marcy as the Peak_name. Here's the SQL:

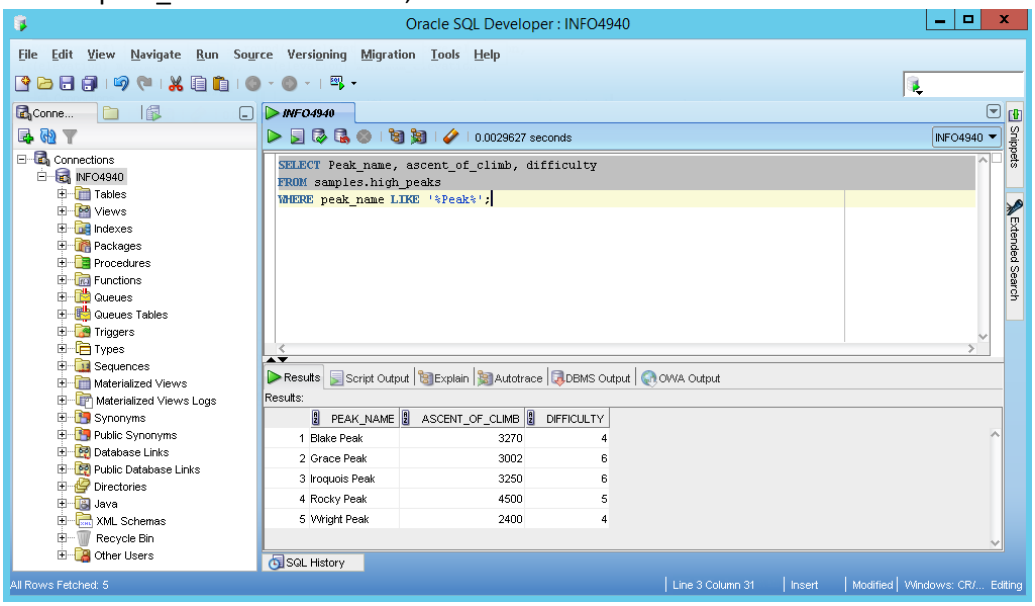
SQL Lab 1 – The Basics (requires access to the samples.high_peaks table)

```
SELECT Peak_name, ascent_of_climb, difficulty
FROM samples.high_peaks
WHERE peak_name = 'Marcy';
```



The result set returns a single value for 'Marcy'. An explicit filter works well, but you must have a perfect match. This time, let's use the LIKE operator, with wildcards, to find any peak names with Peak in their name. Now edit your WHERE clause to: peak_name LIKE '%Peak%' and execute:

```
SELECT Peak_name, ascent_of_climb, difficulty
FROM samples.high_peaks
WHERE peak_name like '%Peak%';
```

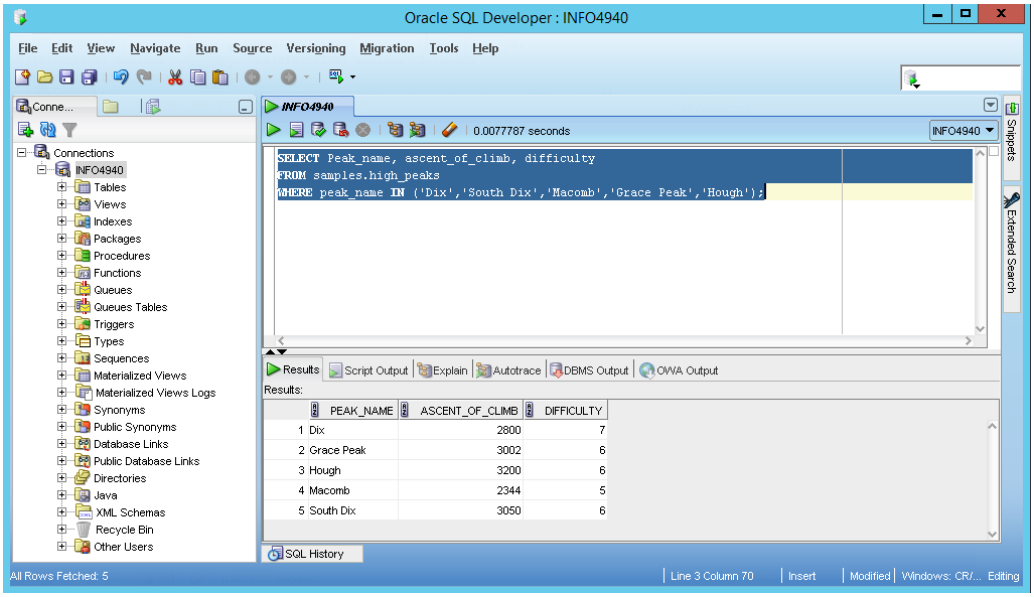


There are 5 peaks with 'Peak' in their name.

SQL Lab 1 – The Basics (requires access to the samples.high_peaks table)

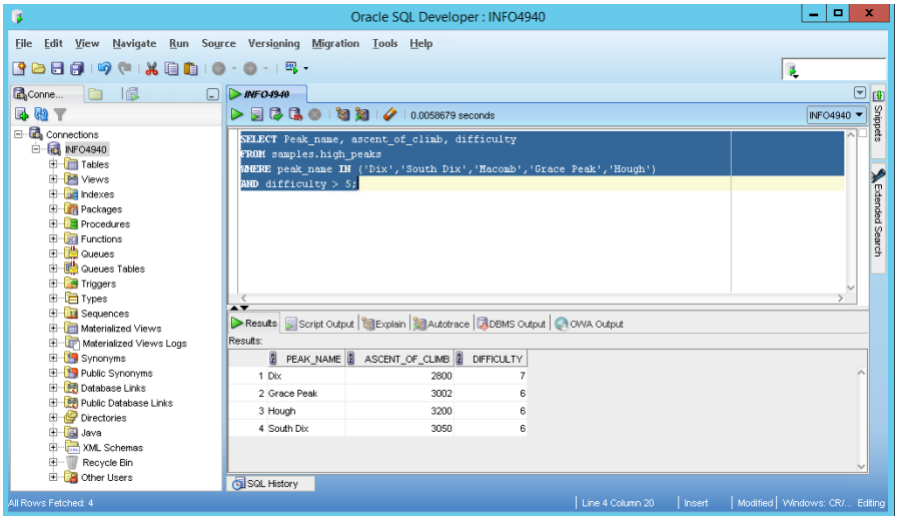
Next, I would like to see data about the 5 peaks which make up the Dix range. An easy way to do this is with an IN list operator. Now change your WHERE clause to an IN operator, listing the 5 peaks, and execute.

```
SELECT Peak_name, ascent_of_climb, difficulty
FROM samples.high_peaks
WHERE peak_name IN ('Dix','South Dix','Macomb','Grace Peak','Hough');
```



Let's add a second filter to our WHERE clause. I want to only display peaks from the Dix range with a difficulty rating over 5. Alter your query by adding: AND difficulty > 5; then execute.

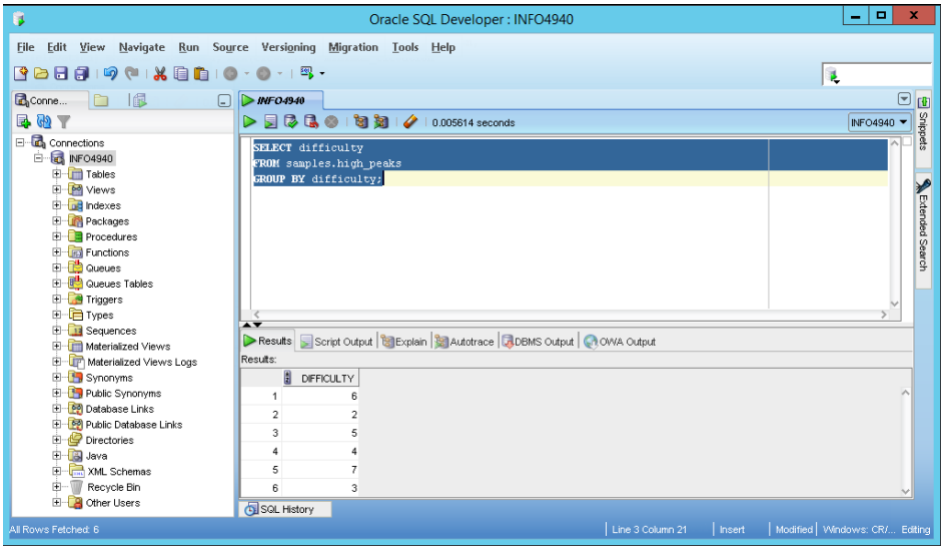
```
SELECT Peak_name, ascent_of_climb, difficulty
FROM samples.high_peaks
WHERE peak_name IN ('Dix','South Dix','Macomb','Grace Peak','Hough')
AND difficulty > 5;
```



SQL Lab 1 – The Basics (requires access to the samples.high_peaks table)

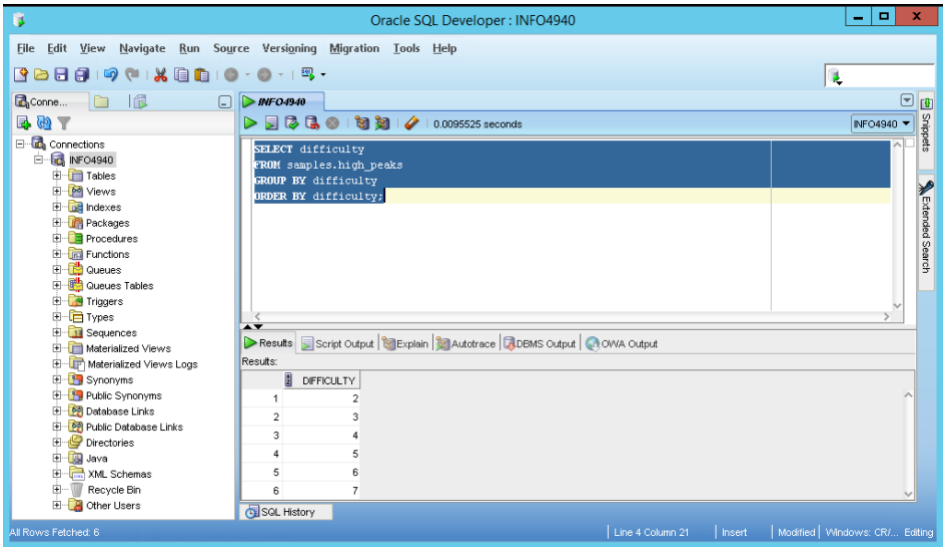
Sometimes you will need to aggregate your data. To aggregate, you will use the GROUP BY clause. We will start with a very simple GROUP BY query of the high_peaks table. We will GROUP BY the difficulty rating. This will give us a list of unique difficulty ratings found in the table. Run the below SQL.

```
SELECT difficulty
FROM samples.high_peaks
GROUP BY difficulty;
```



This result set shows 6 unique values, but they are in random order. Add an ORDER BY clause below the GROUP, to sort by difficulty rating.

```
SELECT difficulty
FROM samples.high_peaks
GROUP BY difficulty
ORDER BY difficulty;
```

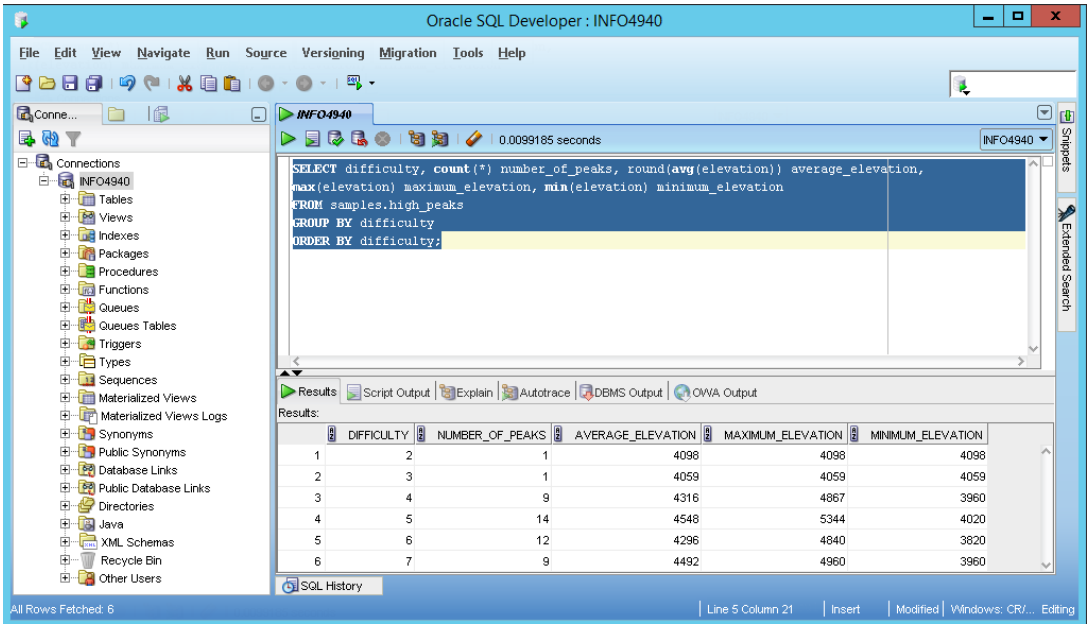


Now you can easily see there are values from 2-7 for difficulty.

SQL Lab 1 – The Basics (requires access to the samples.high_peaks table)

Let's use some of the functions we used earlier in the lab, to learn more about the peaks found in these difficulty ratings. Add the functions: count(), avg(), max(), min() to your current query and execute.

```
SELECT difficulty, count(*) number_of_peaks, round(avg(elevation)) average_elevation,
max(elevation) maximum_elevation, min(elevation) minimum_elevation
FROM samples.high_peaks
GROUP BY difficulty
ORDER BY difficulty;
```



Now you have summarized, aggregated data by difficulty rating.

*** End of SQL Exercise 1